

Metodología de implementación del emprendimiento

Este documento explica **qué metodología** se utilizó para construir y poner en marcha Dolarito (divise.), **cómo se ejecutó** en la práctica y **por qué** es la adecuada para un micro-emprendimiento de software con un equipo reducido, presupuesto acotado y entregas académicas periódicas.

1. Metodología elegida

Desarrollo ágil: SCRUM simplificado (sprints semanales), combinado con una **planificación por fases** del proyecto (relevamiento → diseño → desarrollo → testing → despliegue → documentación). Se elige un **enfoque híbrido** porque permite:

- **Demos incrementales** cada semana frente al docente (que oficia de Product Owner), tomando correcciones al vuelo.
- **Priorización continua** de funcionalidades: las cotizaciones en tiempo real, los favoritos, las alertas y la seguridad (2FA) se agregaron en orden de valor para el usuario.
- **Despliegue continuo**: cada avance se versiona en Git, se hace push a GitHub y la plataforma **Render** publica automáticamente una nueva versión en producción.
- **Trabajo en iteraciones cortas** compatibles con las horas disponibles del equipo (estudiantes) y con el cronograma del Gantt (10 de marzo – 1 de octubre).

¿Por qué no Waterfall puro? En un producto con datos de mercado cambiantes y con un cliente-usuario (docente) que evalúa en revisiones periódicas, el modelo en cascada (que parte de un requerimiento cerrado y no admite cambios hasta el final) hubiera congelado requisitos que efectivamente cambiaron durante el año. Igualmente se conservó de Waterfall el **rigor documental**: cada etapa dejó artefactos (MER/DER, DFD, diccionario de datos, pantallas, ambiente de desarrollo y producción), que es exactamente el material que esta carpeta funcional organiza.

2. Cómo se organizó el trabajo (nuestra adaptación)

Elemento ágil	Cómo lo aplicamos
Sprint (iteración)	Sprints de 1 semana; cada uno termina con una versión funcional y versionada (tag/commit).
Daily	Encuestas y registros diarios de avance (planilla Dailys.xlsx en PRSC), y puesta en común al inicio de cada clase.

Review / demo	Revisiones del docente como demos: las observaciones quedaron documentadas (Revision 24-04 y Revision 08-05) y se transformaron en tareas del siguiente sprint (p. ej., corregir el MR, agregar parámetros a las funciones del UML y crear las dailys).
Backlog	Lista priorizada en la planilla de trabajo: épicas → features → user stories con criterios de aceptación.
Retrospectiva	Al cerrar cada entregable se revisó qué faltó (testing, documentación, presupuesto) y se incorporó a la siguiente iteración.

3. Fases de la implementación

Fase	Entregable	Herramienta
1. Relevamiento	Entrevistas (cerrada y abierta), estadísticas, factibilidad	Google Forms / Docs, planillas
2. Análisis y diseño	MER ↔ DER, diccionario de datos, diagramas (contexto, DFD 0/1/2), pantallas	draw.io, Word, PDF
3. Desarrollo (sprints)	S1 esquema y registros → S2 login/cotizaciones → S3 favoritos/alertas/historial → sprints de seguridad (2FA, RLS, criptos)	Visual Studio Code, Git + GitHub
4. Testing QA/QC	Casos de prueba (test sets) y verificación manual	Planilla Excel de testing, navegador, cURL/Postman
5. Despliegue	Ambiente de producción publicado	Render (CI/CD sobre github main)
6. Documentación y lanzamiento	Manual de usuario, soporte, README, contrato, presupuesto, tag v1.0.0	HTML → PDF (Edge), Markdown

4. Requerimientos según el estándar de la cátedra

Utilizamos el esquema **Épicas → Features → User stories → Criterios de aceptación**.

Ejemplo 1 · Épica: Seguridad de cuenta

Feature: 2FA por correo electrónico

User story: *"Como usuario registrado, quiero recibir un código de 6 dígitos por email al iniciar sesión para que mi cuenta esté protegida aunque roben mi contraseña."*

Criterios de aceptación: **F** (funcional): al activar 2FA, el login exige el código enviado. **NF** (no funcional): el email llega en menos de 1 minuto. **Error**: código incorrecto o expirado → mensaje claro y permiso de reintentar.

Ejemplo 2 · Épica: Cotizaciones en tiempo real

Feature: Actualización automática

User story: *"Como usuario de la app, quiero que las cotizaciones se actualicen solas cada 15 segundos para tomar decisiones con el precio más actualizado."*

Criterios de aceptación: **F**: el precio cambia sin recargar la página. **NF**: la hora de "última actualización" aparece por tarjeta. **Error**: si la API falla, se mantiene el último valor y se avisa.

Ejemplo 3 · Épica: Conveniencia de uso

Feature: Monedas favoritas

User story: "Como usuario frecuente, quiero marcar divisas en favoritos para verlas al iniciar la app sin buscarlas."

Criterios de aceptación: **F**: toggling de estrella agrega/quita el favorito. **NF**: el estado persiste entre sesiones.

Error: un favorito ya existente no se duplica.

5. Justificación de la metodología

- **Por el tamaño del equipo:** 4 integrantes con roles solapados (desarrollo + QA + documentación). Una metodología liviana evita burocracia y maximiza el código producido por persona.
- **Por el dominio del producto:** un mercado de cotizaciones cambia minuto a minuto; Agile permite ajustar fuentes de datos y prioridades sin re-planificar todo el año.
- **Por el presupuesto:** herramientas *free tier* verificadas (Vite, Node, PostgreSQL/Supabase, Git, Render) permiten mantener el costo cerca de \$0 mientras se valida el producto.
- **Por el control de calidad:** QA como actividad continua (y no final): cada sprint termina con un **check** manual/automático; los errores llegan a producción cada vez menos (práctica QA proactiva, no solo QC reactivo).
- **Por la entrega académica:** las fechas del Gantt (10 marzo–1 octubre) y los dos entregables (6 de mayo y 17 de junio) exigen hitos intermedios verificables — exactamente lo que produce un plan de sprints.
- **Por la calidad percibida:** la demo funcional final (recuperación de contraseña, monedas con símbolo y formato «puntos y comas», navegación móvil) es el resultado directo de iterar con el feedback de las revisiones.

6. Herramientas del ambiente de desarrollo virtual

Capas	Tecnología	Rol
Lenguaje (front)	JavaScript / JSX con Vite + React 18	SPA responsive (web + mobile)
Lenguaje (back)	Node.js + Express	API REST, autenticación, notificaciones
Base de datos y motor	PostgreSQL (Supabase)	Persistencia, RLS, migraciones versionadas
Autenticación	JWT + bcrypt + códigos por email	Sesión, 2FA, verificación y recuperación
Diseño en la app	CSS propio + íconos (ui/Icon)	Identidad naval-dorada «divise.»
Tests / verificación	CDP (Edge headless), cURL, scripts Node	Chequeo funcional de producción

7. Ambiente de producción

- **Web:** aplicación publicada en <https://dolarito.onrender.com> (deploy automático desde la rama `main`).
- **Mobile:** mismo frontend responsivo con navegación inferior en teléfonos.
- **Desktop:** diseño a pantalla completa con menú lateral.

- **Datos en vivo:** APIs públicas (dolarapi.com, open.er-api.com, Binance/CoinGecko, currency-api) con sincronización programada y múltiples proveedores de respaldo.

8. Riesgos y mitigación

Riesgo	Mitigación aplicada
API de cotizaciones caída o limitada	Varios proveedores con <i>fallback</i> y formato estable.
Pérdida de código	Git local + respaldo remoto en GitHub (cada commit es un punto de restauración).
Seguridad (password, 2FA, RLS)	bcrypt, JWT con expiración, códigos de 6 dígitos con vencimiento, Row Level Security por usuario.
Retraso en entregas	Sprints cortos + Gantt + hitos con el docente; los fixes críticos (demo) se priorizan sobre features nuevas.